

IBM Bob-Shell Documentation

Tables of Contents

Welcome	1
Install and setup	2
Best practices for using Bob	4
Security guidelines	6
Configuring Bob-Shell	9
IDE integration	15
Starting and using Bob-Shell in interactive mode	19
Starting and using Bob-Shell in non-interactive mode	21
Using slash commands in Bob-Shell	23
Keyboard shortcuts	27
Using custom instructions in Bob-Shell	29
Ignoring files	31
Checkpointing	34
Memory import	36
MCP in Bob-Shell	41
Examples of using Bob-Shell	44
Uninstalling Bob-Shell	46
FAQ	46

Welcome to Bob

Bob is an AI-powered coding assistant that helps you write better code faster. Working directly in your editor, Bob understands your codebase and helps you accomplish development tasks with greater efficiency and confidence. Think of Bob as your helpful coding companion—ready to tackle complex problems so you can focus on what matters most: building great software and enjoying the development process along the way.

What you can do with Bob:

Write and modify code

- Generate complete, production-ready code from natural language descriptions
- Refactor existing code to improve readability, performance, or structure
- Debug issues with intelligent error analysis and suggested fixes
- Create new files, functions, and entire projects with proper structure

Bob's tip: The more specific your request, the better the results. "Create a React component that displays a sortable table of user data" works better than "Make me a table component."

Understand your codebase

Bob helps demystify complex code, turning those "what were they thinking?" moments into clear understanding.

- Get answers to questions about your code's functionality and architecture
- Receive explanations of complex code segments in plain language
- Navigate large codebases more efficiently with contextual understanding
- Learn new frameworks and libraries through interactive guidance

Automate development tasks

Say goodbye to generic commit messages and hello to documentation that actually reflects your code—Bob handles the tedious parts so you can focus on the creative work that actually matters.

- Generate commit messages based on your staged changes
- Create pull request descriptions that clearly explain your changes
- Automate repetitive coding tasks with natural language instructions
- Write and update documentation that stays in sync with your code

Key features

Specialized modes

Bob adapts to your specific needs with purpose-built modes—think of them as different hats Bob wears depending on what you need:

- **Code mode:** Write, modify, and refactor code with precision
- **Ask mode:** Get answers and explanations about your codebase
- **Plan mode:** Plan and design before implementation
- **Advanced mode:** Access extended capabilities, such as MCP, for complex tasks

Each mode optimizes Bob's behavior for different development scenarios—like having the right tool for every job. No need to adjust your communication style when your needs change.

Powerful tools

Bob comes with a comprehensive set of tools that extend far beyond simple text generation—these are what make Bob truly special:

- **File operations:** Read, write, and modify files in your project
- **Command execution:** Run commands directly from the chat interface
- **Browser control:** Navigate and interact with web resources
- **Code analysis:** Search codebases and analyze definitions
- **MCP integration:** Connect to external tools and services

These tools work together seamlessly, allowing you to accomplish complex tasks without constantly switching between applications. Less context-switching means more productive coding sessions and fewer interruptions to your flow state.

MCP (Model Context Protocol)

Model Context Protocol (MCP) extends Bob's capabilities by allowing you to add custom tools that supercharge your workflow:

- Connect to databases and APIs
- Access specialized development resources
- Integrate with your organization's internal systems
- Create custom workflows for your specific needs

MCP is available in all modes except Code mode.

Getting started

To get started with Bob:

1. Review the best practices guide to get a brief understanding on how to use Bob effectively. For more information, see [Best practices for using Bob](#).
2. Consult the security guidelines to minimize potential security risks. For more information, see [Security guidelines](#).

Bob's tip: Start with something you're currently working on. Bob learns best when helping with real problems rather than hypothetical scenarios.

Installing and setting up Bob-Shell

Bob-Shell brings Bob's AI capabilities directly to your terminal, allowing you to interact with Bob through a command-line interface. This guide walks you through the installation process and initial setup.

System requirements

- Node.js version 20 or later
- One of the following operating systems: macOS, Linux, Windows

Installation

Install Bob-Shell using the appropriate command for your operating system:

macOS, Linux, and Windows

Run this command in your terminal:

```
npm install -g "https://ibm.biz/bob-shell-install"
```

Authentication

To use Bob-Shell, you need to authenticate with your Bob API key:

1. Locate the API key included in your private preview invitation.
2. Start Bob-Shell by running the `bob` command in your terminal.
3. When prompted, paste your API key into the terminal.

Bob-Shell stores your API key securely for future sessions, so you only need to enter it once.

File references

You can reference files in your prompts using the `@` symbol, just like in Bob's interactive mode:

```
bob -p "Explain the code in @src/main.js"
```

Safety features

By default, Bob-Shell only uses "non-destructive" tools (e.g., "read file") in non-interactive mode. To enable writing and updating files, specify `--yolo` when you invoke Bob-Shell:

```
bob -p "Fix the bugs in @src/main.js" --yolo
```

Note: Even with `yolo` mode enabled, Bob-Shell will **not** write or update files outside the directory where Bob-Shell was started.

Processing output

The output contains both Bob-Shell's answer along with its "thinking" steps. To make subsequent processing of the result easier, add an instruction like *Enclose the answer in markdown tags*:

```
bob -p "What Java version is this application for? Use the info in @pom.xml.  
Enclose the answer in markdown tags" > ../bobresp.txt
```

Example result:

```
I'll check the Java version specified in the pom.xml file:I need to provide the  
absolute path to read the pom.xml file:  
```markdown  
This application is built for Java 8, as specified in the pom.xml file with the
following compiler settings:
<maven.compiler.source>1.8</maven.compiler.source>
<maven.compiler.target>1.8</maven.compiler.target>
```

## ## Next steps

- [Starting and using Bob-Shell in interactive mode] (start-bobshell-interactive.html)
- [Starting and using Bob-Shell in non-interactive mode] (start-bobshell-non-interactive.html)
- [Using slash commands in Bob-Shell] (slash-commands-bobshell.html)

---

# Best practices for using Bob

This guide provides essential best practices to help you get the most out of Bob. Whether you're new to Bob or looking to optimize your workflow, these recommendations will help you work more effectively with your AI coding assistant.

---

## Understanding and using modes

Bob's modes are specialized personas that tailor Bob's behavior for specific tasks. Selecting the right mode for each task is crucial for optimal results.

### Built-in modes

Bob comes with several built-in modes, each designed for specific purposes:

Mode	Purpose	When to use
<b>Code</b>	Writing and modifying code	For implementing features, fixing bugs, or making code improvements
<b>Advanced</b>	Complex development tasks	When you need full access to all tools, including MCP and browser capabilities
<b>Plan</b>	Planning and designing	Before implementation, when you need to plan architecture or create technical specifications
<b>Ask</b>	Getting information	When you need explanations or information without modifying files

### Mode selection strategy

For the most effective workflow:

1. **Start with Plan mode** for new projects or complex features to create a detailed implementation plan before writing any code.
2. **Switch to Code mode** for most day-to-day development tasks when you're ready to implement.
3. **Use Ask mode** when you need explanations or information without modifying files.
4. **Switch to Advanced mode** when you need access to additional tools like browser automation or MCP servers.

---

## Workflow optimization

### Plan before coding

Always start complex projects or features in Plan mode to:

- Generate a detailed implementation plan

- Break down complex problems into manageable steps
- Identify potential challenges before they arise
- Create a roadmap for implementation

This approach prevents breaking changes and provides a clear direction for development.

## Use checkpoints effectively

Checkpoints automatically version your workspace files during Bob tasks, enabling:

- Safe experimentation with AI-suggested changes
- Easy recovery from undesired modifications
- Comparison of different implementation approaches

**Bob's tip:** If Bob starts producing subpar work, use the "Restore Files & Task" checkpoint option to roll back to a previous state. This is more effective than trying to correct flawed output with new instructions.

To use a Checkpoint to restore your work to a previous state:

1. Scroll up in your chat interface to locate a Checkpoint you want to restore from.
2. Click the "Restore Checkpoint" button, then click "Restore Files and Task", then "Confirm".

## Integrate with version control

Regularly commit and push changes to your version control system:

- Create frequent, small commits with clear messages
- Push changes to remote repositories regularly
- Use branches for experimental features

Git provides an essential safety net to prevent accidental, permanent deletion of code.

# Communication strategies

---

## Write effective prompts

The quality of your prompts directly affects the quality of Bob's responses:

- **Be specific and clear:** Vague prompts lead to vague outputs. Detail what Bob should and should not do.
- **Provide examples:** When possible, include examples of the desired output format or style.
- **Use the Enhance Prompt feature:** Click the enhance button to automatically improve your query with additional context and structure.

## Use context mentions

Context mentions let you reference specific elements of your project directly in your conversations with Bob:

- Use `@/path/to/file.js` to include specific file contents
- Use `@/path/to/folder` to include all files in a directory
- Use `@problems` to include VS Code Problems panel diagnostics
- Use `@terminal` to include recent terminal output

This approach is more efficient than copying and pasting code or describing file locations.



## Manage the context window

To prevent Bob from mixing up your goals:

- Start new tasks regularly with specific aims
- Avoid giving Bob your entire codebase at once
- Use direct file references to provide targeted context
- Break complex tasks into smaller, focused subtasks

## Security and control

---

### Configure auto-approval settings

Bob allows you to control which actions require your approval:

- **Manual approval:** Review and approve every action Bob takes (safest setting)
- **Auto-approve:** Grant Bob the ability to run specific tasks without interruption
- **Hybrid approach:** Auto-approve low-risk actions while requiring confirmation for riskier tasks

Configure these settings based on your comfort level and the sensitivity of your project.

### Use `.bobignore`

The `.bobignore` file lets you specify files and directories that Bob should not access or modify:

1. Create a `.bobignore` file in your project root
2. Add patterns for sensitive files, build artifacts, and large assets
3. Use the same syntax as `.gitignore`

This helps protect sensitive information and prevent accidental changes to generated files.

For more detailed information on security guidelines, see [Security guidelines](#).

### Set up rules

Bob allows you to specify rules to control how Bob performs tasks:

- Define rules globally or per project in the `.bob` directory
- Create mode-specific rules in `.Bob/rules-{mode-slug}/` directories
- Use rules to enforce coding standards, documentation requirements, or other workflow constraints

---

## Security guidelines

Bob is a tool that provides significant capabilities for code development and system interaction. With these capabilities comes the responsibility to use Bob securely. This guide outlines essential security practices for you when working with Bob.

## Security checklist

---

- Configure `.bobignore` to restrict file access
- Review and limit auto-approve settings
- Handle secrets securely

- Use MCP with proper authentication and encryption
- Review Bob's output before implementing

## File access restrictions

---

You can control Bob's file access by configuring which directories and file types it can interact with. The `.bobignore` file uses the same syntax as `.gitignore` and should be one of the first security measures you implement when setting up Bob.

### Setting up `.bobignore`

1. Create a `.bobignore` file in your workspace
2. Add patterns for sensitive files and directories. Include patterns for any non-approved data types.

```
Example .bobignore patterns
.env
secrets/
*.key
config/credentials.json
```

Bob actively monitors the `.bobignore` file, and any changes are automatically applied. For detailed information, see [Using `.bobignore` to Control File Access](#).

### Understanding limitations

While `.bobignore` effectively controls Bob's access through its tools, it has some important limitations:

- It only applies to files within your current workspace
- Some write operations might bypass restrictions
- It doesn't create a system-level sandbox

Review the complete [key limitations and scope](#) to understand how `.bobignore` protects your files.

## Auto-approve settings

---

With Bob, you can automatically approve various actions without confirmation prompts. While this speeds up your workflow, it significantly increases security risks.

⚠ **SECURITY WARNING:** Auto-approve settings bypass confirmation prompts, giving Bob direct access to your system. This can result in data loss, file corruption, or worse. Command line access is particularly dangerous, as it can potentially execute harmful operations.

### High-risk auto-approve settings

Setting	Risk	Recommendation
Edit files	High	Enable only in controlled environments
Execute commands	High	Use whitelist and avoid wildcards
Use MCP servers	Medium-High	Only with trusted servers
Read files	Medium	Consider sensitive data exposure

Always review Bob's output to ensure it's accurate and that any generated code will act as intended. Never inherently trust output from any AI system.

For detailed information on each setting and its security implications, see [Auto-Approving Actions](#).

## Handling secrets securely

---

Never provide secrets directly to any AI system, including Bob. Even temporary inclusion of secrets in code can lead to unintended exposure.

### Best practices for secrets management

- Store secrets in environment variable files
- Ensure both `.gitignore` and `.bobignore` restrict access to files that store secrets
- Use secret management tools when possible
- Follow the principle of least privilege when assigning credentials - only grant the minimum permissions necessary for each task.

### Delegation of permissions

AI systems should not leverage credentials that allow them to act on your behalf without your intervention and review. When AI systems need to act on your behalf:

- Use delegation mechanisms like OAuth
- Implement time-limited tokens
- Monitor and audit all actions

## Using MCP securely

---

Model Context Protocol (MCP) extends Bob's functionality by connecting to external tools and services. While powerful, MCP connections require careful security consideration.

### MCP architecture overview

MCP uses a client-server architecture:

- Bob acts as the host containing the MCP client
- The client connects to MCP servers (local or remote)
- Servers provide additional tools and capabilities

### Security requirements for MCP servers

When using MCP servers, practice the following guidelines:

- **Authentication:** Verify the identity of users accessing the server
- **Encryption:** Secure data in transit between Bob and the server
- **Access controls:** Limit what actions the server can perform
- **Auditing:** Track all actions for accountability

Remote MCP servers must adhere to the same security requirements as any traditional server infrastructure, including endpoint protection, network restrictions, and proper access controls.

For shared MCP servers, ensure proper auditability and accountability so actions can be traced.

## Additional security considerations

---

- **Review generated content:** Always verify Bob's output before implementing it
- **Regular updates:** Keep Bob and its dependencies updated
- **Security training:** Ensure team members understand AI security risks

- **Incident response:** Have a plan for addressing potential security incidents
- **Workspace isolation:** Consider using dedicated workspaces for sensitive projects
- **Permission scoping:** Limit Bob's access to only what's needed for specific tasks

By following these security best practices, you can leverage Bob's powerful capabilities while maintaining a secure development environment.

## Configuring Bob-Shell

Bob-Shell offers extensive configuration options to customize your experience. This guide explains how to configure Bob-Shell to match your workflow and preferences.

## Getting started

You can configure Bob-Shell using:

1. **Project-specific settings:** Create `.bob/settings.json` in your project directory
2. **User-wide settings:** Edit `~/.bob/settings.json` in your home directory
3. **Command-line arguments:** Pass options like `--model` or `--sandbox` when starting Bob-Shell

# Example: Start Bob-Shell with a specific model and sandbox mode enabled  
`bob --model pro-latest --sandbox`

## Configuration system

### How configuration works

Bob-Shell uses a layered configuration system where settings from different sources are combined according to a specific precedence order:

Priority	Source	Location	Scope
7 (Highest)	Command-line arguments	<code>bob --option value</code>	Current session
6	Environment variables	Shell or <code>.env</code> files	System/session
5	System settings file	<code>/etc/bob-shell/settings.json</code>	All users, all projects
4	Project settings file	<code>.bob/settings.json</code>	Current project
3	User settings file	<code>~/.bob/settings.json</code>	Current user, all projects
2	System defaults file	<code>/etc/bob-shell/system-defaults.json</code>	All users, all projects
1 (Lowest)	Hardcoded defaults	Built into Bob-Shell	Always applied

When the same setting is defined in multiple places, the higher-priority source takes precedence.

### Settings file locations

Bob-Shell looks for settings files in these locations:

- **Project settings:** `.bob/settings.json` in your project directory
- **User settings:** `~/.bob/settings.json` in your home directory
- **System settings:**

- Linux: `/etc/bob-shell/settings.json`
- Windows: `C:\ProgramData\bob-shell\settings.json`
- macOS: `/Library/Application Support/Bob-Shell/settings.json`
- **System defaults:**
  - Linux: `/etc/bob-shell/system-defaults.json`
  - Windows: `C:\ProgramData\bob-shell\system-defaults.json`
  - macOS: `/Library/Application Support/Bob-Shell/system-defaults.json`

**Tip:** You can reference environment variables in your settings files using `$VAR_NAME` or `${VAR_NAME}` syntax. For example: `"apiKey": "$MY_API_TOKEN"`.

## Core settings categories

Bob-Shell settings are organized into categories. Each category contains related settings that control specific aspects of Bob-Shell's behavior.

### General settings

Control basic Bob-Shell behavior and preferences.

```
{
 "general": {
 "preferredEditor": "code",
 "vimMode": false,
 "disableAutoUpdate": false,
 "disableUpdateNag": false,
 "checkpointing": {
 "enabled": true
 }
 }
}
```

Setting	Type	Default	Description
<code>preferredEditor</code>	string	<code>undefined</code>	Editor to use when opening files
<code>vimMode</code>	boolean	<code>false</code>	Enable Vim keybindings
<code>disableAutoUpdate</code>	boolean	<code>false</code>	Prevent automatic updates
<code>disableUpdateNag</code>	boolean	<code>false</code>	Hide update notifications
<code>checkpointing.enabled</code>	boolean	<code>false</code>	Enable <a href="#">session checkpointing</a>

### UI settings

Customize Bob-Shell's appearance and interface elements.

```
{
 "ui": {
 "theme": "GitHub",
 "hideBanner": true,
 "hideTips": false,
 "showLineNumbers": true
 }
}
```

Setting	Type	Default	Description
<code>theme</code>	string	<code>undefined</code>	UI color theme
<code>customThemes</code>	object	<code>{}</code>	Custom theme definitions
<code>hideWindowTitle</code>	boolean	<code>false</code>	Hide window title bar

Setting	Type	Default	Description
<code>hideTips</code>	boolean	<code>false</code>	Hide helpful tips
<code>hideBanner</code>	boolean	<code>false</code>	Hide application banner
<code>hideFooter</code>	boolean	<code>false</code>	Hide footer
<code>showMemoryUsage</code>	boolean	<code>false</code>	Show memory usage stats
<code>showLineNumbers</code>	boolean	<code>false</code>	Show line numbers in chat
<code>showCitations</code>	boolean	<code>false</code>	Show citations for generated text
<code>accessibility.disableLoadingPhrases</code>	boolean	<code>false</code>	Disable loading phrases

## Model settings

Configure AI model behavior and performance.

```
{
 "model": {
 "name": "pro-latest",
 "maxSessionTurns": 20,
 "summarizeToolOutput": {
 "run_shell_command": {
 "tokenBudget": 2000
 }
 },
 "chatCompression": {
 "contextPercentageThreshold": 0.7
 }
 }
}
```

Setting	Type	Default	Description
<code>name</code>	string	<code>undefined</code>	Model to use for conversations
<code>maxSessionTurns</code>	number	<code>-1</code>	Maximum conversation turns (-1 = unlimited)
<code>summarizeToolOutput</code>	object	<code>undefined</code>	Tool output summarization settings
<code>chatCompression.contextPercentageThreshold</code>	number	<code>0.7</code>	Chat compression threshold (0-1)
<code>skipNextSpeakerCheck</code>	boolean	<code>false</code>	Skip next speaker check

## Context settings

Control how Bob-Shell manages project context and memory.

```
{
 "context": {
 "fileName": ["CONTEXT.md", "AGENTS.md"],
 "discoveryMaxDirs": 200,
 "includeDirectories": ["../shared-lib", "~/reference-code"],
 "fileFiltering": {
 "respectGitIgnore": true,
 "respectBobIgnore": true
 }
 }
}
```

Setting	Type	Default	Description
<code>fileName</code>	string/array	<b>undefined</b>	Context file name(s)
<code>importFormat</code>	string	<b>undefined</b>	Memory import format
<code>discoveryMaxDirs</code>	number	200	Max directories to search
<code>includeDirectories</code>	array	<code>[]</code>	Additional directories to include
<code>loadFromIncludeDirectories</code>	boolean	<b>false</b>	Load context from included dirs
<code>fileFiltering.respectGitIgnore</code>	boolean	<b>true</b>	Honor .gitignore files
<code>fileFiltering.respectBobIgnore</code>	boolean	<b>true</b>	Honor .bobignore files
<code>fileFiltering.enableRecursiveFileSearch</code>	boolean	<b>true</b>	Enable recursive file search

## Tools settings

Configure how Bob-Shell uses and manages tools.

```
{
 "tools": {
 "sandbox": "docker",
 "allowed": ["run_shell_command(git)", "run_shell_command(npm test)"],
 "exclude": ["write_file"]
 }
}
```

Setting	Type	Default	Description
<code>sandbox</code>	boolean/string	<b>undefined</b>	Sandbox execution environment
<code>usePty</code>	boolean	<b>false</b>	Use node-pty for shell commands
<code>core</code>	array	<b>undefined</b>	Restrict built-in tools (allowlist)
<code>exclude</code>	array	<b>undefined</b>	Tools to exclude from discovery
<code>allowed</code>	array	<b>undefined</b>	Tools that bypass confirmation
<code>discoveryCommand</code>	string	<b>undefined</b>	Command for tool discovery
<code>callCommand</code>	string	<b>undefined</b>	Command for calling tools

## MCP settings

Configure Model Context Protocol server connections.

```
{
 "mcpServers": {
 "mainServer": {
 "command": "bin/mcp_server.py"
 },
 "remoteServer": {
 "url": "https://example.com/mcp",
 "headers": {
 "Authorization": "Bearer token123"
 }
 }
 }
}
```

Setting	Type	Default	Description
<code>mcp.serverCommand</code>	string	<b>undefined</b>	Command to start MCP server

Setting	Type	Default	Description
<code>mcp.allowed</code>	array	<code>undefined</code>	Allowlist of MCP servers
<code>mcp.excluded</code>	array	<code>undefined</code>	Denylist of MCP servers
<code>mcpServers.&lt;SERVER_NAME&gt;</code>	object	<code>•</code>	Server-specific configuration

For each MCP server, you can configure:

- **command**: Command to execute (for local servers)
- **args**: Command-line arguments
- **env**: Environment variables
- **cwd**: Working directory
- **url**: SSE endpoint URL (for remote servers)
- **httpUrl**: HTTP endpoint URL (for remote servers)
- **headers**: HTTP headers for requests
- **timeout**: Request timeout in milliseconds
- **trust**: Trust server and bypass confirmations
- **includeTools**: Tool names to include
- **excludeTools**: Tool names to exclude

## Command-line arguments

Pass these arguments when starting Bob-Shell to override settings for that session:

```
Start Bob-Shell with specific settings
bob --model pro-latest --sandbox --approval-mode auto_edit
```

Argument	Description	Example
<code>--model, -m</code>	Specify model	<code>bob -m pro-latest</code>
<code>--prompt, -p</code>	Non-interactive prompt	<code>bob -p "Explain this code"</code>
<code>--prompt-interactive, -i</code>	Interactive initial prompt	<code>bob -i "Help me debug"</code>
<code>--sandbox, -s</code>	Enable sandbox mode	<code>bob -s</code>
<code>--debug, -d</code>	Enable debug mode	<code>bob -d</code>
<code>--all-files, -a</code>	Include all files as context	<code>bob -a</code>
<code>--yolo</code>	Auto-approve all tool calls	<code>bob --yolo</code>
<code>--approval-mode</code>	Set tool approval mode	<code>bob --approval-mode=auto_edit</code>
<code>--allowed-tools</code>	Tools to auto-approve	<code>bob --allowed-tools="git status"</code>
<code>--checkpointing</code>	Enable checkpointing	<code>bob --checkpointing</code>
<code>--include-directories</code>	Add directories to workspace	<code>bob --include-directories=../lib</code>

## Context files

Context files (like `AGENTS.md`) provide instructions to the AI model. These files are loaded hierarchically:

1. **Global context**: `~/ .bob/AGENTS.md` (applies to all projects)
2. **Project context**: `AGENTS.md` in project root and parent directories
3. **Local context**: `AGENTS.md` in subdirectories (for component-specific instructions)

### Example context file

```
Project: My TypeScript Library
```



## ## General instructions

- Follow existing coding style
- Add JSDoc comments to all functions
- Prefer functional programming patterns
- Target TypeScript 5.0 and Node.js 20+

## ## Coding style

- Use 2 spaces for indentation
- Interface names should be prefixed with ``I``
- Private class members should be prefixed with ``_``
- Use strict equality (``===`` and ``!==``)

## Managing context

- Use `/memory refresh` to reload all context files
- Use `/memory show` to view the current context

# Sandboxing

---

Sandboxing provides security when running potentially unsafe operations:

```
Enable sandboxing for a session
bob --sandbox
```

You can create custom sandbox environments:

1. Create `.bob/sandbox.Dockerfile` in your project
2. Base it on the `bob-shell-sandbox` image
3. Add your custom dependencies

```
FROM bob-shell-sandbox
```

```
Add custom dependencies
```

```
RUN apt-get update && apt-get install -y python3-dev
```

Build and use your custom sandbox:

```
BUILD_SANDBOX=1 bob -s
```

# Usage statistics

---

Bob-Shell collects anonymous usage statistics to improve the product. This includes:

- Tool usage patterns (names, success/failure, duration)
- API request metrics (model, duration, success)
- Session configuration information

No personal information, prompt content, or file content is collected.

To opt out, add this to your settings:

```
{
 "privacy": {
 "usageStatisticsEnabled": false
 }
}
```

# IDE integration with Bob-Shell

Connect Bob-Shell directly to your code editor for a seamless development experience. This integration enhances Bob-Shell's capabilities by providing real-time workspace awareness and enabling powerful features like in-editor diff viewing.

## Overview

Feature	Description	Benefit
Workspace awareness	Bob-Shell sees your recent files and cursor position	More contextually relevant assistance
Native diff viewing	Review code changes in your editor's diff tool	Easier code review and modification
Command integration	Access Bob-Shell features from editor commands	Streamlined workflow
Seamless file editing	Accept changes directly in your editor	Faster implementation of suggestions

**Note:** Currently, Bob-IDE and compatible editors are supported. Additional editor support may be added in future releases.

## Getting started

1. Start Bob-Shell from your Bob-IDE terminal
2. Accept the automatic integration prompt when it appears
3. Use Bob-Shell normally - it now has access to your workspace context
4. When Bob-Shell suggests code changes, they'll appear in Bob-IDE's diff viewer

## Benefits of IDE integration

### Enhanced context awareness

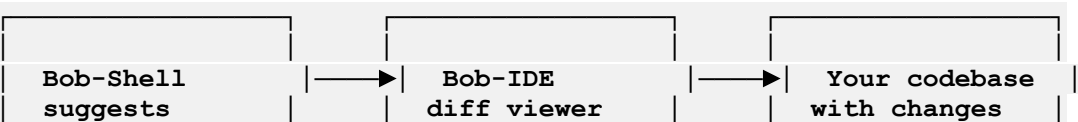
Without IDE integration, Bob-Shell only knows about files you explicitly reference. With integration enabled, Bob-Shell automatically gains:

- Access to your **10 most recently accessed files**
- Knowledge of your **current cursor position**
- Visibility of your **selected text** (up to 16KB)

This contextual awareness allows Bob-Shell to provide more relevant assistance without requiring you to manually reference files.

### Seamless code modifications

When Bob-Shell suggests code changes:





This workflow allows you to:

- Review changes in your familiar editor environment
- Edit the suggested changes before accepting them
- Accept or reject changes using standard editor commands
- Keep your hands on the keyboard throughout the process

## Installation methods

Choose the installation method that works best for you:

### Method 1: Automatic setup (recommended)

When you run Bob-Shell inside Bob-IDE's integrated terminal, it automatically detects the environment and offers to set up integration:

```
Bob-Shell has detected you're running in Bob-IDE.
Would you like to enable IDE integration?

[Yes] [No] [Don't ask again]
```

Selecting Yes will install Bob-Shell and configure your connection settings.

### Method 2: Command-line installation

To run Bob-IDE from Bob-Shell by typing code, add it the \$PATH environment variable:

Open the Command Palette (Cmd+Shift+P), type 'shell command', and run: `Install 'code' command in PATH command`.

Restart the terminal for the new \$PATH value to apply.

You can now type 'code .' in any folder to start editing files in that folder.

## Managing the connection

### Connection commands

Command	Purpose	Example
<code>/ide enable</code>	Activate the IDE connection	<code>/ide enable</code>
<code>/ide disable</code>	Deactivate the IDE connection	<code>/ide disable</code>
<code>/ide status</code>	Check connection status and context	<code>/ide status</code>
<code>/ide install</code>	Install the companion extension	<code>/ide install</code>

### Checking connection status

To verify your connection and see what context Bob-Shell has access to:

```
/ide status
```

Example output:

```
Connected to Bob-IDE
Recent files:
- src/components/Header.tsx
- src/utils/api.js
- package.json
```

## Working with code changes

---

When Bob-Shell suggests code modifications, you'll see them in your editor's diff viewer.

### Accepting changes

You have multiple ways to accept suggested changes:

Method	Action
Editor UI	Click the ✓ checkmark in the diff editor's title bar
Keyboard	Press <b>Cmd+S</b> (macOS) or <b>Ctrl+S</b> (Windows/Linux)
Command Palette	Run <b>Bob-Shell: Accept Diff</b>
Bob-Shell CLI	Type <b>yes</b> when prompted

### Rejecting changes

To reject suggested changes:

Method	Action
Editor UI	Click the ✗ icon in the diff editor's title bar
Keyboard	Close the diff editor tab
Command Palette	Run <b>Bob-Shell: Close Diff Editor</b>
Bob-Shell CLI	Type <b>no</b> when prompted

### Modifying suggestions

You can edit the suggested code directly in the diff editor before accepting it. This allows you to:

- Fix minor issues in the suggestion
- Adapt the code to your specific needs
- Add comments or additional logic
- Ensure the changes match your coding style

**Tip:** For frequently repeated changes, select "Yes, allow always" in Bob-Shell to auto-accept similar changes in the future.

## Advanced usage

---

### Bob-IDE commands

Access Bob-Shell features directly from Bob-IDE's Command Palette (**Cmd+Shift+P** or **Ctrl+Shift+P**):

Command	Function
<b>Bob-Shell: Run</b>	Start a new Bob-Shell session in the terminal
<b>Bob-Shell: Accept Diff</b>	Accept changes in the active diff editor

Command	Function
<b>Bob-Shell: Close Diff Editor</b>	Reject changes and close the diff editor
<b>Bob-Shell: View Third-Party Notices</b>	Show third-party notices

## Using with sandboxed environments

### macOS sandboxing

When using Bob-Shell with macOS Seatbelt sandboxing:

- The IDE integration requires network access
- Use a Seatbelt profile that allows network connections
- The default `permissive-open` profile works with IDE integration

### Docker containers

When running Bob-Shell in a Docker container:

- The integration can still connect to Bob-IDE on your host machine
- Bob-Shell automatically finds the IDE server via `host.docker.internal`
- Ensure your Docker networking allows container-to-host connections

## Troubleshooting

### Common issues and solutions

Issue	Possible Causes	Solution
Connection fails	Extension not installed or running	Install extension and run <code>/ide enable</code>
Directory mismatch	Bob-Shell running outside workspace	<code>cd</code> to your workspace directory
Connection lost	Network issue or IDE restart	Run <code>/ide enable</code> to reconnect
No workspace open	Missing workspace in IDE	Open a folder/workspace in your IDE

### Error message reference

#### Connection errors

**Disconnected: Failed to connect to IDE companion extension in [IDE Name]**

##### Solution:

1. Verify the Bob-Shell Companion extension is installed and enabled
2. Open a new terminal in your IDE
3. Run `/ide enable`

**Disconnected: IDE connection error. The connection was lost unexpectedly**

##### Solution:

1. Run `/ide enable` to reconnect
2. If the issue persists, restart your IDE

#### Configuration errors

**Disconnected: Directory mismatch**

**Solution:**

1. Navigate to the same directory that's open in your IDE
2. Restart Bob-Shell

**Disconnected: To use this feature, please open a workspace folder**

**Solution:**

1. Open a folder or workspace in your IDE
2. Restart Bob-Shell

**General errors**

**IDE integration is not supported in your current environment**

**Solution:** Run Bob-Shell from within a supported IDE's integrated terminal

**No installer is available for IDE**

**Solution:** Install the Bob-Shell Companion extension manually from your IDE's marketplace

## Best practices

---

- **Start Bob-Shell from your IDE's terminal** for automatic integration
- **Keep your workspace focused** on relevant files for better context
- **Review diffs carefully** before accepting changes
- **Use keyboard shortcuts** for faster workflow
- **Check connection status** if Bob-Shell seems to be missing context

---

## Starting and using Bob-Shell in interactive mode

Interactive mode provides a conversational interface to Bob directly in your terminal, allowing real-time assistance with your development tasks.

## Getting started

---

1. Open a new terminal window.
2. Navigate to the main directory of your project.
3. To start Bob-Shell in interactive mode, run:

**bob**



## Basic usage

### Interacting with Bob-Shell

- Type your instructions or questions directly in the terminal.
- Press Enter to send your message to Bob.
- Bob-Shell will respond with its analysis and suggestions.
- For tool usage (like reading or writing files), you'll be prompted to approve or decline each action.

### Referencing files

Use the @ symbol to reference files in your project:

**Explain the functionality in @src/main.js**

This tells Bob-Shell to read and analyze the specified file before responding.

## Using slash commands

Type / to access a menu of available commands:

- Built-in commands like **/help** for assistance
- Mode-switching commands like **/code** or **/ask**
- Custom commands you've created

For a complete list of available commands, see [Using slash commands in Bob-Shell](#).

## Viewing file changes

When Bob-Shell needs to modify files, it will show you the proposed changes:

- By default, changes are displayed in the terminal with a CLI diff view
- To use an external editor for reviewing changes:
  1. Configure your preferred editor with the **/editor** command
  2. Select the "Show diff in editor" option when prompted to review changes

## Advanced features

---

### Tool approvals

For security, Bob-Shell requires your approval before:

- Reading files
- Writing or modifying files
- Executing commands

You can approve or decline each action individually when prompted.

### Multi-turn conversations

Bob-Shell maintains context throughout your conversation, allowing you to:

- Ask follow-up questions
- Refine previous requests
- Build on earlier responses

## Tips for effective use

---

- Be specific in your requests to get more targeted responses
- Use `@` references to provide code context when needed
- For complex tasks, break them down into smaller steps
- Use slash commands to quickly access common functionality
- When working with large codebases, direct Bob-Shell to the most relevant files

## When to use interactive mode

---

Interactive mode works best for:

- Exploratory coding sessions
- Debugging and troubleshooting
- Learning new concepts or technologies
- Tasks that require multiple back-and-forth exchanges
- Projects where you need to review changes before they're applied

---

## Starting and using Bob-Shell in non-interactive mode

Non-interactive mode allows you to use Bob-Shell directly from the command line without entering an interactive session. This approach works well for automation, scripting, and batch processing tasks.

## Getting started

---

1. Open a new terminal window.
2. Navigate to the main directory of your project.
3. Run Bob-Shell with your prompt using the `-p` command.

For example:



```
bob -p "Explain this project"
```

## Basic usage

---

### Providing prompts to Bob-Shell

You can specify your prompt directly with the `-p` command:

```
bob -p "Explain this project"
```

### Pipe content as input

You can pipe text content to Bob-Shell:

```
cat buildError.txt | bob -p "Explain this build error"
```

### Save results to a file

Redirect the output to save results:

```
bob -p "Review @bigFile.java" > review.md
```

### Reference project files

Use the `@` symbol to reference files in your project:

```
bob -p "Summarize the functionality in @src/main.js"
```

## Advanced options

---

### Enable file modifications

By default, Bob-Shell only uses non-destructive tools (like reading files) in non-interactive mode. To enable writing and updating files, add the `--yolo` flag:

```
bob -p "Fix bugs in @app.js" --yolo
```

**Note:** Even with the `--yolo` flag enabled, Bob-Shell will not write or update files outside the directory where it was started.

### Format output for processing

The output contains both Bob-Shell's answer and its thinking steps. For easier processing, add instructions to format the output:

```
bob -p "What Java version is this application using? Check @pom.xml. Enclose the answer in markdown tags" > analysis.md
```

## When to use non-interactive mode

---

Non-interactive mode works best for:

- Integrating Bob-Shell into automation scripts or CI/CD pipelines
- Processing multiple files with a single command
- Getting quick insights without starting an interactive session
- Generating documentation from code

## Tips for effective use

- For complex tasks that might require multiple tool uses, interactive mode usually works better.
- When processing large files or projects, be specific about which files to analyze.
- Use structured output instructions (like "Format the output as JSON") for easier parsing in scripts.
- Consider creating shell aliases or scripts for frequently used Bob-Shell commands.
- For multi-line prompts, save them to a file and pipe to Bob-Shell:

```
cat prompt.txt | bob
```

## Using slash commands in Bob-Shell

Slash commands are fully compatible with Bob-Shell. You can create custom slash commands to automate repetitive tasks and extend functionality with simple markdown files.

To get started, type / in Bob-Shell to see all available commands, or create your own by adding a markdown file to `.bob/commands/` or `~/ .bob/commands/`.

## Why use slash commands?

Slash commands provide several key benefits:

- **Workflow automation:** Turn complex multi-step processes into single commands
- **Team standardization:** Share commands across your team for consistent practices
- **Context preservation:** Include project-specific context in every command
- **Quick access:** Fuzzy search and autocomplete for instant command discovery

## How slash commands work

When you type / in Bob-Shell, a menu appears showing all available commands. These commands come from two sources:

Command Type	Source	Purpose
Custom workflow commands	<code>.bob/commands/</code> or <code>~/ .bob/commands/</code>	User-created automation for specific tasks
Mode commands	Built-in modes	Switch Bob's operational context

## Creating custom commands

Custom commands extend Bob's functionality by adding markdown files to specific directories:

Location	Scope	Path
Project-specific	Available in current project only	<code>.bob/commands/</code> in your workspace root
Global	Available in all projects	<code>~/ .bob/commands/</code> in your home directory

The filename becomes the command name. For example:

```
.bob/commands/
└─ review.md → /review
```

```
| test-api.md → /test-api
| deploy-check.md → /deploy-check
```

## Command name processing

When creating commands through the UI, command names are automatically processed:

- Converted to lowercase
- Spaces replaced with dashes
- Special characters removed
- Leading/trailing dashes removed

Example: "My Cool Command!" becomes `my-cool-command`

## Basic command format

Create a simple command by adding a markdown file:

```
Help me review this code for security issues and suggest improvements.
```

## Advanced command with frontmatter

Add metadata using frontmatter for enhanced functionality:

```

description: Create a new API endpoint
argument-hint: <endpoint-name> <http-method>

```

```
Create a new API endpoint called $1 that handles $2 requests.
Include proper error handling and documentation.
```

## Frontmatter fields

Field	Purpose	Example
<code>description</code>	Appears in the command menu	"Create a new API endpoint"
<code>argument-hint</code>	Shows expected arguments	" "

# Command management in Bob-Shell

Bob-Shell supports the same slash commands as Bob-IDE. While Bob-Shell does not provide a dedicated UI for managing commands, you can:

1. Create command files manually in the `.bob/commands/` directory in your project or `~/ .bob/commands/` in your home directory
2. Edit existing command files with any text editor
3. Use the same command files across both Bob-Shell and the Bob VS Code extension

# Using slash commands

Type `/` in Bob-Shell to see a unified menu containing both types of commands:

1. **Unified menu:** Both custom commands and mode-switching commands appear together
2. **Autocomplete:** Start typing to filter commands (e.g., `/sam` shows `sample-command-name`)
3. **Fuzzy search:** Find commands even with partial matches
4. **Description preview:** See command descriptions in the menu

```
/mode code Switch to Code mode
/mode ask Switch to Ask mode
/review Review code for security issues
/api-endpoint <endpoint-name> <http-method>
```

## Argument hints

Argument hints provide instant help for slash commands, showing you what kind of information to provide when a command expects additional input.

When you type / to bring up the command menu, commands that expect arguments will display a light gray hint next to them. This hint tells you what kind of argument the command is expecting.

For example:

- `/mode <mode_slug>` - The hint `<mode_slug>` indicates you should provide a mode name like `code` or `debug`
- `/api-endpoint <endpoint-name> <http-method>` - Shows you need both an endpoint name and HTTP method

After selecting the command, it will be inserted into the chat input followed by a space. The hint is not inserted; it is only a visual guide to help you know what to type next. You must then manually type the argument after the command.

### Adding argument hints to custom commands

You can add argument hints to your custom commands using the `argument-hint` field in the frontmatter:

```

description: Create a new API endpoint
argument-hint: <endpoint-name> <http-method>

```

Create a new API endpoint called \$1 that handles \$2 requests.  
Include proper error handling and documentation.

This will display as `/api-endpoint <endpoint-name> <http-method>` in the command menu.

### Best practices for argument hints

- **Be specific:** Use descriptive placeholders like `<file-path>` instead of generic ones like `<arg>`
- **Show multiple arguments:** If your command needs multiple inputs, show them all: `<source> <destination>`
- **Use consistent format:** Always wrap placeholders in angle brackets: `<placeholder>`
- **Keep it concise:** Hints should be brief and clear

### Common questions about arguments

Question	Answer
What if I don't provide the argument?	The command might not work as expected, or it might prompt you for more information. The hint is there to help you get it right the first time.
Do all commands have hints?	No, only commands that are designed to take arguments will have hints. Commands that work without additional input won't show hints.
Can I use a command without replacing the hint?	The hint text (like <code>&lt;mode_slug&gt;</code> ) needs to be replaced with actual values. Leaving the hint text will likely cause the command to fail or behave unexpectedly.

## Best practices

---

### Command naming

- Use descriptive, action-oriented names
- Keep names concise but clear
- Use hyphens for multi-word commands
- Avoid generic names like `help` or `test`
- Note: Names are automatically slugified (lowercase, special characters removed)
- The `.md` extension is automatically added/removed as needed

### Command content

- Start with a clear directive
- Use structured formats (lists, sections)
- Include specific requirements
- Reference project conventions
- Keep commands focused on a single task

### Organization

- Group related commands in subdirectories
- Use consistent naming patterns
- Document complex commands
- Version control your commands
- Share team commands in the project repository

## Troubleshooting

---

### Commands not appearing

If your commands aren't showing up in the menu:

- **Check file location:** Ensure custom command files are in `.bob/commands/` or `~/ .bob/commands/`
- **Verify file extension:** Custom commands must be `.md` files

### Command not found

When a slash command isn't found, the LLM will see:

The slash command  `'/unknown-command'` was not found. Please check the command name and try again.

### Command conflicts

- Custom project commands override global custom commands with the same name
- Use unique names to avoid conflicts
- When creating duplicate names through the UI, numbers are appended (e.g., `new-command-1`, `new-command-2`)

## Cross-platform compatibility

---

Slash commands work identically across both Bob-Shell and Bob-IDE. This means:

- Commands created in one environment can be used in the other
- The same command syntax and behavior applies in both environments
- Team-shared commands work consistently regardless of which interface team members use

---

## Keyboard shortcuts for Bob-Shell

Master Bob-Shell's keyboard shortcuts to dramatically improve your productivity. This guide organizes all available shortcuts by function, with platform-specific variations where applicable.

### Quick reference

---

Category	Shortcut	Purpose
<a href="#">Navigation</a>	Ctrl+A, Ctrl+E, Arrow keys	Move through text and history
<a href="#">Editing</a>	Ctrl+K, Ctrl+U, Ctrl+W	Modify and edit text
<a href="#">Application control</a>	Ctrl+C, Ctrl+D, Ctrl+L	Control Bob-Shell's behavior
<a href="#">Special features</a>	Ctrl+Y, Ctrl+T, Ctrl+G	Access advanced functionality

### Navigation shortcuts

---

Efficiently move through text and command history:

#### Cursor movement

Shortcut	macOS Alternative	Description
Ctrl+A / Home		Move cursor to beginning of line
Ctrl+E / End		Move cursor to end of line
Ctrl+B / Left Arrow		Move cursor one character left
Ctrl+F / Right Arrow		Move cursor one character right
Ctrl+Left Arrow	Option+Left Arrow	Move cursor one word left
Ctrl+Right Arrow	Option+Right Arrow	Move cursor one word right

#### History navigation

Shortcut	Alternative	Description
Up Arrow	Ctrl+P	Navigate up through input history
Down Arrow	Ctrl+N	Navigate down through input history

### Editing shortcuts

---

Efficiently edit and modify text:

#### Text deletion

Shortcut	Description
Ctrl+H / Backspace	Delete character to left of cursor
Ctrl+D / Delete	Delete character to right of cursor

Shortcut	Description
<b>Ctrl+W / Ctrl+Backspace</b>	Delete word to left of cursor
<b>Ctrl+Delete</b>	Delete word to right of cursor
<b>Ctrl+U</b>	Delete from cursor to beginning of line
<b>Ctrl+K</b>	Delete from cursor to end of line

## Text manipulation

Shortcut	Description
<b>` ` + Enter</b>	Insert a newline (when at end of line)
<b>Ctrl+V</b>	Paste clipboard content (text or image)
<b>Ctrl+X / Alt+Enter</b>	Open current input in external editor
<b>Tab</b>	Autocomplete current suggestion
<b>Esc</b> (double press)	Clear the input prompt

# Application control

---

Control Bob-Shell's core functionality:

## Session management

Shortcut	Description
<b>Ctrl+C</b>	Cancel ongoing request and clear input Press twice to exit application
<b>Ctrl+D</b>	Exit application (when input is empty) Press twice to confirm
<b>Ctrl+L</b>	Clear the screen
<b>Esc</b>	Close dialogs and suggestions

## Mode toggles

Shortcut	Description
<b>!</b> (empty prompt)	Toggle shell mode
<b>Ctrl+Y</b>	Toggle auto-approval (YOLO mode) for all tool calls
<b>Ctrl+O</b>	Toggle debug console display
<b>Ctrl+S</b>	Toggle response truncation Allow long responses to print fully
<b>Ctrl+T</b>	Toggle tool descriptions display

# Special features

---

Access specialized functionality:

## Suggestions and selections

Shortcut	Description
<b>Tab / Enter</b>	Accept selected suggestion
<b>Up Arrow</b>	Navigate up through suggestions
<b>Down Arrow</b>	Navigate down through suggestions

## Radio button selection

Shortcut	Description
Up Arrow / k	Move selection up
Down Arrow / j	Move selection down
Enter	Confirm selection
1-9	Select item by number
Multi-digit numbers	Press digits in quick succession for items >9

## IDE integration

Shortcut	Description
Ctrl+G	View context received from IDE

# Platform-specific notes

---

## macOS users

On macOS, many shortcuts use the **Option** key (also called **Alt** or **⌘**) instead of **Ctrl** for word-based operations:

- Use **Option+Left/Right** instead of **Ctrl+Left/Right** for word movement
- Use **Option+Backspace** instead of **Ctrl+W** for deleting the previous word

## Terminal compatibility

Some shortcuts may be intercepted by your terminal emulator before reaching Bob-Shell:

- If **Ctrl+C** doesn't work as expected, try **Ctrl+C** twice
- If **Ctrl+Left/Right** doesn't work, check your terminal's keyboard settings

# Keyboard shortcut tips

---

- **Learn incrementally:** Start with basic navigation (**Ctrl+A**, **Ctrl+E**, arrow keys)
- **Practice deletion shortcuts:** **Ctrl+U** and **Ctrl+K** are particularly useful
- **Use history navigation:** **Up/Down** arrows save time when repeating commands
- **Try shell mode:** Type **!** at an empty prompt to quickly run shell commands
- **Customize your terminal:** Some terminals allow remapping keys for better compatibility

# Using custom instructions in Bob-Shell

Custom instructions personalize how Bob-Shell behaves in your terminal environment. You provide specific guidance that shapes responses, coding style, and decision-making processes when using Bob-Shell.

# What are custom instructions?

---

Custom instructions define specific behaviors, preferences, and constraints beyond Bob-Shell's basic capabilities. These instructions help Bob-Shell understand your preferences when working in the terminal environment.



Examples include:

- Coding style preferences
- Documentation standards
- Testing requirements
- Workflow guidelines
- Project-specific conventions

## Setting up custom instructions for Bob-Shell

---

Bob-Shell uses the same custom instructions system as the Bob IDE. You can define instructions at different levels:

### Workspace-wide instructions

To create custom instructions that apply to all modes when using Bob-Shell in any project directory:

1. Create a `.bob/rules/` directory in your project root
2. Add markdown (`.md`), text (`.txt`), or XML (`.xml`) files containing your instructions
3. Bob-Shell reads these files in alphabetical order by filename

Here's an example directory structure:

```
.
├── .bob/
│ └── rules/ # Workspace-wide rules
│ ├── 01-general.md
│ └── 02-coding-style.txt
└── ... (other project files)
```

### Mode-specific instructions

You can also set instructions that apply only when Bob-Shell is using a specific mode:

1. Create a directory named `.bob/rules-{modeSlug}/` in your project root (e.g., `.bob/rules-code/` for Code mode)
2. Add markdown files with your instructions
3. Bob-Shell reads these files in alphabetical order by filename

## How instructions are combined

---

When you use Bob-Shell, it combines instructions in this order:

1. Mode-specific instructions (from `.bob/rules-{modeSlug}/`)
2. Workspace-wide instructions (from `.bob/rules/`)

This order ensures that more specific instructions (mode-level) take precedence over more general ones (workspace-wide).

## Rules about custom instruction files

---

- **File location:** Store files within `.bob/rules/` and `.bob/rules-{modeSlug}/` directories
- **File naming:** Use a numbering prefix (e.g., `01-`, `02-`) to control the order

# Examples for Bob-Shell use cases

When using Bob-Shell in your terminal, custom instructions can be particularly helpful for:

- "Always use relative paths when suggesting file operations in the terminal"
- "Prefer using standard Unix commands over specialized tools when possible"
- "Format shell script examples with proper error handling"
- "When suggesting commands, include explanations of what each flag does"
- "Always check for file existence before operations in shell scripts"
- "Use portable shell syntax that works across bash, zsh, and other common shells"

## Team standardization

When working in team environments, using the `.bob/rules/` directory structure under version control standardizes Bob-Shell's behavior across your team. This ensures consistent:

- Code style
- Documentation practices
- Command-line conventions
- Development workflows

By keeping custom instructions in your project repository, you ensure that all team members using Bob-Shell receive the same guidance, regardless of their individual settings.

## Ignoring files with .bobignore

Control which files Bob-Shell can access by creating a `.bobignore` file in your project. This feature helps you exclude sensitive, irrelevant, or large files from Bob-Shell's operations, improving performance and security.

## Overview

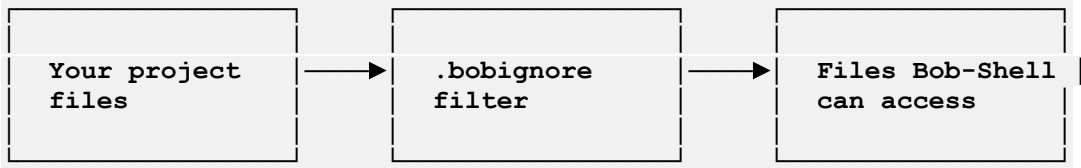
Feature	Description	Benefit
Path exclusion	Prevent Bob-Shell from accessing specific files or directories	Protect sensitive information
Pattern matching	Use glob patterns to match multiple files	Efficiently exclude file types
Negation support	Override exclusions for specific files	Fine-grained control
Project-specific	Each project can have its own exclusion rules	Customized to project needs

## Why use .bobignore?

- **Privacy protection:** Keep sensitive files like API keys and credentials private
- **Performance improvement:** Exclude large binary files or directories that slow down operations
- **Noise reduction:** Remove irrelevant files from Bob-Shell's context
- **Focus control:** Direct Bob-Shell's attention to the most relevant parts of your codebase

## How .bobignore works

When you add paths to your **.bobignore** file, Bob-Shell tools that respect this file will automatically exclude matching files and directories from their operations. For example, when using the **read\_many\_files** command, any paths in your **.bobignore** file will be skipped.



**Note:** Changes to your **.bobignore** file require restarting your Bob-Shell session to take effect.

## Pattern syntax

The **.bobignore** file follows the same pattern syntax as **.gitignore**:

Pattern	Description	Example
<code>file.txt</code>	Matches a specific file	<code>apikeys.txt</code>
<code>*.ext</code>	Matches all files with the extension	<code>*.log</code>
<code>/dir/</code>	Matches a directory	<code>/node_modules/</code>
<code>/path/to/file</code>	Anchors the path to the root	<code>/src/config/secrets.json</code>
<code>!pattern</code>	Negates a previous pattern	<code>!important.md</code>
<code>#</code>	Comment line	<code># API credentials</code>

## Getting started

### Creating a .bobignore file

1. Create a file named **.bobignore** in the root of your project directory
2. Add patterns for files and directories you want to exclude
3. Save the file and restart your Bob-Shell session

```
From your project root
touch .bobignore
echo "# Files to ignore" > .bobignore
echo "secrets/" >> .bobignore
```

### Common exclusion patterns

Here are some common patterns you might want to add to your **.bobignore** file:

```
Sensitive information
.env
secrets/
password
credential
apikey

Large directories
node_modules/
.git/
dist/
```

```
build/

Binary and media files
*.zip
*.tar.gz
*.mp4
*.jpg
*.png

Log files
*.log
logs/
```

## Pattern examples

---

### Excluding specific directories

To exclude entire directories and their contents:

```
Exclude the packages directory and all its contents
/packages/

Exclude all node_modules directories anywhere in the project
**/node_modules/

Exclude the dist directory but only at the root level
/dist/
```

### Using wildcards

Wildcards let you match multiple files with similar patterns:

```
Exclude all markdown files
*.md

Exclude all JavaScript files in the src directory
/src/**/*.js

Exclude all files with "test" in their name
test
```

### Using negation

You can override exclusions for specific files:

```
Exclude all markdown files
*.md

But include README.md
!README.md

And include all markdown files in the docs directory
!docs/*.md
```

## Advanced usage

---

### Combining patterns

You can create sophisticated exclusion rules by combining patterns:

```
Exclude all JavaScript files
*.js

But include all files in the src directory
!src/**/*.js

Except for test files in the src directory
src/**/*.test.js
```

## Debugging your .bobignore file

If Bob-Shell seems to be ignoring files you want it to access, or accessing files you want it to ignore:

1. Check your `.bobignore` file for conflicting patterns
2. Remember that more specific patterns (like negations) should come after general patterns
3. Restart your Bob-Shell session after making changes
4. Use absolute paths if relative paths aren't working as expected

## Best practices

---

- **Start simple:** Begin with a few essential patterns and add more as needed
- **Comment your patterns:** Add comments to explain why certain files are excluded
- **Be specific:** Use precise patterns to avoid accidentally excluding important files
- **Consider security:** Always exclude files containing sensitive information
- **Maintain regularly:** Update your `.bobignore` file as your project evolves

---

# Checkpointing in Bob-Shell

## Overview

---

Bob-Shell's checkpointing feature creates automatic snapshots of your project before applying changes, providing a safety net for your development process. This powerful feature lets you experiment confidently with AI-suggested modifications, knowing you can easily revert to a previous state if needed.

## Why use checkpointing?

---

- **Safety net:** Experiment with code changes without fear of breaking your project
- **Easy rollback:** Restore your project to a previous state with a single command
- **Conversation preservation:** Return to the exact conversation context when a change was made
- **Non-intrusive:** Works alongside your existing Git workflow without interference

## How checkpointing works

---

When you approve a file-modifying operation (such as `write_file` or `replace`), Bob-Shell automatically:

1. **Creates a Git snapshot** in a shadow repository (`~/ .bob/history/<project_hash>`)

**Note:** This shadow repository is separate from your project's Git repository and won't interfere with your normal Git workflow.

2. **Saves your conversation history** up to that point

3. **Records the tool call** that was about to be executed

This three-part checkpoint allows you to:

- Revert all files to their previous state
- Resume the conversation from where you were
- Review and potentially re-execute the original tool call

All checkpoint data is stored locally on your machine:

- Git snapshots in `~/.bob/history/<project_hash>`
- Conversation history and tool calls in `~/.bob/tmp/<project_hash>/checkpoints`

## Enabling checkpointing

---

Checkpointing is disabled by default. You can enable it in two ways:

### Method 1: Command-line flag (per session)

To enable checkpointing for a single session:

```
bob --checkpointing
```

### Method 2: Settings file (persistent)

To enable checkpointing for all sessions:

1. Edit your `settings.json` file
2. Add the following configuration:

```
{
 "general": {
 "checkpointing": {
 "enabled": true
 }
 }
}
```

## Managing checkpoints

---

### Viewing available checkpoints

To see all saved checkpoints for your current project:

```
/restore
```

Bob-Shell will display a list of checkpoint files with names that include:

- Timestamp
- Modified file name
- Tool that was used

Example: `2025-06-22T10-00-00_000Z-my-file.txt-write_file`

### Restoring a checkpoint

To restore your project to a specific checkpoint:

```
/restore <checkpoint_file>
```

For example:

```
/restore 2025-06-22T10-00-00_000Z-my-file.txt-write_file
```

This will:

1. Revert your files to their previous state
2. Restore the conversation history
3. Re-propose the original tool call

## Best practices

---

- **Enable for critical work:** Always use checkpointing when working on important code
- **Test before relying:** Try the restore functionality on a non-critical project first
- **Regular commits:** Continue making regular Git commits for meaningful changes
- **Checkpoint cleanup:** Periodically remove old checkpoints you no longer need
- **Combine with version control:** Use checkpointing alongside your normal Git workflow for maximum safety

## Limitations

---

- Checkpointing only tracks files that are already under version control or have been modified by Bob-Shell
- External changes made outside of Bob-Shell won't be included in checkpoints
- Very large projects may experience slightly slower checkpoint creation times

---

## Memory import processor

Create modular, maintainable context files by importing content from multiple sources. The memory import processor allows you to break down large AGENTS.md files into smaller, reusable components using a simple `@file.md` syntax.

## Overview

---

Feature	Description	Benefit
File imports	Include content from other files	Organize complex context files
Path flexibility	Support for relative and absolute paths	Work with any file structure
Nested imports	Files can import other files	Create hierarchical structures
Safety features	Circular import detection and security checks	Prevent common problems

## Why use memory imports?

---

- **Maintainability:** Break large context files into logical, focused components
- **Reusability:** Share common instructions across multiple projects
- **Organization:** Create a clear hierarchy of context information
- **Collaboration:** Enable team members to work on different parts of context files

- **Versioning:** Track changes to individual components more effectively

## Basic usage

---

### Import syntax

Use the @ symbol followed by the path to the file you want to import:

```
Main AGENTS.md file

This is the main content.

@./components/instructions.md

More content here.

@./shared/configuration.md
```

### Supported path formats

Path Format	Example	Description
Same directory	@./file.md	Import from current directory
Parent directory	@../file.md	Import from parent directory
Subdirectory	@./components/file.md	Import from nested directory
Absolute path	@/absolute/path/to/file.md	Import using full path

## Working with imports

---

### Basic import example

```
My AGENTS.md

Welcome to my project!

@./getting-started.md

Features

@./features/overview.md
```

### Nested imports

Files can import other files, creating a hierarchical structure:

```
main.md

@./header.md
@./content.md
@./footer.md
```

Where `header.md` might contain:

```
Project Header

@./shared/title.md
```

This creates a tree structure of imports:



```
main.md
├── header.md
│ └── shared/title.md
├── content.md
└── footer.md
```

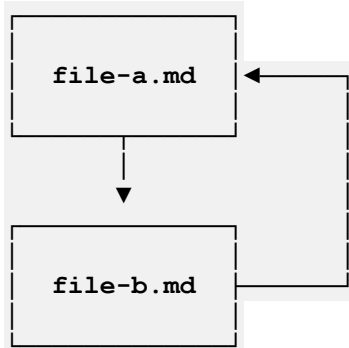
## Safety features

---

The memory import processor includes several safety mechanisms to prevent common issues:

### Circular import detection

The processor automatically detects and prevents circular imports:



If `file-a.md` imports `file-b.md` and `file-b.md` tries to import `file-a.md`, the processor will detect this circular reference and prevent it.

### File access security

The `validateImportPath` function ensures that imports are only allowed from specified directories, preventing access to sensitive files outside the allowed scope.

### Maximum import depth

To prevent infinite recursion, there's a configurable maximum import depth (default: 5 levels). This means imports can be nested up to 5 levels deep.

## Error handling

---

The processor handles common errors gracefully:

### Missing files

If a referenced file doesn't exist, the import will fail with an error comment in the output:

```
<!-- Error importing ./non-existent-file.md: File not found -->
```

### File access errors

Permission issues or other file system errors are handled with appropriate error messages:

```
<!-- Error importing ./protected-file.md: Permission denied -->
```

## Advanced features

---

## Code region detection

The import processor uses the **marked** library to detect code blocks and inline code spans, ensuring that `@` imports inside these regions are properly ignored:

```
My document
```

```
This is a real import:
@./real-import.md
```

```
```markdown
# Code example
This is not a real import:
@./fake-import.md
```

Import tree visualization

The processor generates an import tree that shows the hierarchy of imported files:

```
Memory Files └── project: AGENTS.md └── components/instructions.md | └──
components/common/header.md └── shared/configuration.md └── shared/settings/defaults.md
```

This tree preserves the order of imports and shows the complete import chain for debugging purposes.

Comparison with `/memory`` command

Feature	Memory Import Processor	/memory` Command
-----	-----	-----
Structure	Hierarchical tree	Flat, linear document
File boundaries	Seamless integration	Marked with comments
Import syntax	@./path/to/file.md`	N/A (loads all AGENTS.md files)
Control	Fine-grained inclusion	Automatic directory scanning
Primary use	Modular context files	Project-wide context

API reference

Core functions

processImports()

```
```typescript
async function processImports(
 content: string,
 basePath: string,
 debugMode?: boolean,
 importState?: ImportState
): Promise<ProcessImportsResult>
```

Processes import statements in AGENTS.md content.

#### Parameters:

- **content**: The content to process for imports
- **basePath**: The directory path where the current file is located
- **debugMode**: Whether to enable debug logging (default: false)
- **importState**: State tracking for circular import prevention

**Returns:** Object containing processed content and import tree

## validateImportPath()

```
function validateImportPath(
 importPath: string,
 basePath: string,
 allowedDirectories: string[]
) : boolean
```

Validates import paths to ensure they are safe and within allowed directories.

### Parameters:

- **importPath**: The import path to validate
- **basePath**: The base directory for resolving relative paths
- **allowedDirectories**: Array of allowed directory paths

**Returns:** Whether the import path is valid

## findProjectRoot()

```
async function findProjectRoot(startDir: string): Promise<string>
```

Finds the project root by searching for a `.git` directory upwards from the given start directory.

### Parameters:

- **startDir**: The directory to start searching from

**Returns:** The project root directory (or the start directory if no `.git` is found)

## Interface definitions

```
interface ProcessImportsResult {
 content: string; // The processed content with imports resolved
 importTree: MemoryFile; // Tree structure showing the import hierarchy
}

interface MemoryFile {
 path: string; // The file path
 imports?: MemoryFile[]; // Direct imports, in the order they were imported
}
```

# Best practices

---

## Organization strategies

- **Use descriptive file names** that clearly indicate the content's purpose
- **Create a logical folder structure** for your imported files
- **Keep imports shallow** - avoid deeply nested import chains when possible
- **Document your structure** with comments explaining the import hierarchy
- **Use relative paths** when possible for better portability between environments

## Performance considerations

- **Minimize the number of imports** to reduce processing time
- **Keep individual files focused** on specific topics or components
- **Test your imports** regularly to ensure all referenced files exist and are accessible

- **Consider caching** for frequently used but rarely changed imports

## Troubleshooting

---

### Common issues

Issue	Possible Causes	Solution
Import not working	File doesn't exist or path is incorrect	Verify file path and existence
Circular import warnings	Files importing each other	Restructure your imports to avoid cycles
Permission errors	Files not readable	Check file permissions
Path resolution issues	Relative path confusion	Try using absolute paths

### Debugging techniques

Enable debug mode to see detailed logging of the import process:

```
const result = await processImports(content, basePath, true);
```

This will output detailed information about:

- Files being processed
- Import paths being resolved
- Circular references detected
- Error conditions encountered

---

## MCP in Bob-Shell

Bob-Shell supports the Model Context Protocol (MCP), allowing you to extend Bob's capabilities by connecting to external services and tools. This guide explains how to configure and use MCP servers with Bob-Shell.

### What is MCP?

---

An MCP (Model Context Protocol) server acts as a bridge between Bob and external services like databases, APIs, or custom scripts. With MCP, you can extend Bob's capabilities beyond its built-in features.

MCP servers provide Bob with:

- **Tool discovery:** Access to available tools with their descriptions and parameters
- **Tool execution:** The ability to call specific tools with arguments and receive structured responses
- **Resource access:** The capability to read data from specific resources

### Why use MCP?

---

- **Extend functionality:** Add custom capabilities to Bob that aren't available out-of-the-box
- **Integrate with services:** Connect Bob to your databases, APIs, and other systems
- **Automate workflows:** Create specialized tools for your specific needs
- **Share capabilities:** Distribute useful tools across your team

## Configuring MCP servers

---

You can manage MCP server configurations at two levels:

1. **Global configuration:** Settings in `<USER_HOME>/.bob/mcp_settings.json` apply to all workspaces
2. **Project-level configuration:** Settings in `.bob/mcp.json` apply only to the current project

When a server name exists in both global and project configurations, the **project-level configuration takes precedence**.

## Edit MCP settings files

Use Bob-IDE, or another text editor, to modify your MCP settings files.

Refer to the following example:

```
{
 "mcpServers": {
 "server1": {
 "command": "python",
 "args": ["/path/to/server.py"],
 "env": {
 "API_KEY": "your_api_key"
 },
 "alwaysAllow": ["tool1", "tool2"],
 "disabled": false
 }
 }
}
```

## Configuration properties

Each server configuration requires one of these properties:

- **command:** Path to the executable for Stdio transport
- **url:** SSE endpoint URL for remote servers

Optional properties include:

- **args:** Command-line arguments for Stdio transport
- **headers:** Custom HTTP headers for SSE transport
- **env:** Environment variables for the server process
- **cwd:** Working directory for Stdio transport
- **timeout:** Request timeout in milliseconds (default: 60,000ms)
- **alwaysAllow:** Tool names to approve automatically
- **disabled:** Set to **true** to disable the server

## Transport types

---

MCP supports two ways to communicate with servers:

### STDIO transport

STDIO transport runs servers locally on your machine:

- Uses standard input/output streams for communication
- Provides lower latency and better security
- Runs as a child process on your machine

Example configuration:

```
{
 "mcpServers": {
 "local-server": {
 "command": "node",
 "args": ["server.js"],
 "cwd": "/path/to/project/Bob",
 "env": {
 "API_KEY": "your_api_key"
 },
 "alwaysAllow": ["tool1", "tool2"]
 }
 }
}
```

## SSE transport

SSE transport connects to remote servers over HTTP/HTTPS:

- Uses Server-Sent Events protocol
- Works with servers hosted on different machines
- Requires network access

Example configuration:

```
{
 "mcpServers": {
 "remote-server": {
 "url": "https://your-server-url.com/mcp",
 "headers": {
 "Authorization": "Bearer your-token"
 },
 "alwaysAllow": ["tool3"]
 }
 }
}
```

## Platform-specific examples

---

### Windows configuration

```
{
 "mcpServers": {
 "puppeteer": {
 "command": "cmd",
 "args": [
 "/c",
 "npx",
 "-y",
 "@modelcontextprotocol/server-puppeteer"
]
 }
 }
}
```

### macOS and Linux configuration

```
{
 "mcpServers": {
 "puppeteer": {
```

```

 "command": "npx",
 "args": [
 "-y",
 "@modelcontextprotocol/server-puppeteer"
]
 }
}
}

```

## Runtime version manager configuration

When using version managers like [asdf](#) or [mise](#):

```

{
 "mcpServers": {
 "mcp-batchit": {
 "command": "mise",
 "args": [
 "x",
 "--",
 "node",
 "/Users/myself/workspace/mcp-batchit/build/index.js"
],
 "alwaysAllow": [
 "search",
 "batch_execute"
]
 }
 }
}

```

---

## Examples of using Bob-Shell

This document provides practical examples of how to use Bob-Shell for various common tasks. These examples demonstrate effective ways to interact with Bob-Shell in different scenarios.

## Fixing errors in shell commands

---

When you encounter errors in your shell commands, Bob-Shell can help diagnose and fix them:

```

Navigate to your project directory
cd your-project/

```

```

Start Bob-Shell
bob

```

```

Switch to shell mode
> !

```

```

Type in the command that's causing an error
> make build

```

```

When you see an error message like 'No rule to make target `xxx', needed by
`yyy'.'
Ask Bob-Shell for help
> Help me to fix the error

```

## Code explanation and improvement

---

Bob-Shell can help you understand and improve existing code:

```
Start Bob-Shell in your project directory
bob

Ask for an explanation of a specific file
> Explain what @src/utils.js does and how it works

Request improvements to your code
> Review @src/api.js and suggest improvements for error handling
```

## Creating new files and features

---

Use Bob-Shell to help you create new components or features:

```
Start Bob-Shell in your project directory
bob

Ask for help creating a new component
> Create a React component for a user profile page that displays name, email, and
 avatar

Generate a utility function
> Write a utility function that formats dates in YYYY-MM-DD format
```

## Debugging assistance

---

Bob-Shell can help you debug issues in your code:

```
Start Bob-Shell in your project directory
bob

Share error logs with Bob-Shell
> I'm getting this error when running my app: "TypeError: Cannot read property
 'map' of undefined". Here's the relevant code: @src/components/List.js

Or pipe error output directly to Bob-Shell
In your terminal (not in Bob-Shell)
npm run start 2>&1 | bob -p "Help me understand and fix this error"
```

## Documentation generation

---

Generate documentation for your code using Bob-Shell:

```
Start Bob-Shell in your project directory
bob

Generate documentation for a specific file
> Create JSDoc comments for all functions in @src/helpers.js

Generate a README for your project
> Create a README.md file for my project based on the code in @src/index.js
```

## Learning new concepts

---



Use Bob-Shell to learn about new technologies or concepts:

```
Start Bob-Shell
bob

Ask about a specific technology
> Explain how React hooks work and give me examples of useState and useEffect

Learn about design patterns
> Explain the Observer pattern and how I might implement it in JavaScript
```

---

## Uninstalling Bob-Shell

If you need to remove Bob-Shell from your system, you can uninstall it using npm.

### Uninstall command

---

Run the following command in your terminal:

```
npm uninstall -g bobshell
```

This command removes Bob-Shell from your global npm packages.

---

## Frequently Asked Questions

### General

---

#### What is Bob?

Bob is an AI-powered coding companion that can help you write code, refactor existing code, and complete various tasks. Bob's capabilities can be extended with custom instructions.

#### How does Bob work?

Bob uses large language models (LLMs) to understand your requests and translate them into actions. It can:

- Read and write files in your project.
- Run commands in your terminal.
- Browse the internet (if enabled).
- Use external tools via the Model Context Protocol (MCP).

You interact with Bob through a chat interface, where you provide instructions and review/approve its proposed actions.

#### What can Bob do?

Bob can help with a variety of coding tasks, including:

- Generating code from natural language descriptions.
- Refactoring existing code.

- Fixing bugs.
- Writing documentation.
- Explaining code.
- Answering questions about your codebase.
- Automating repetitive tasks.
- Creating new files and projects.

## Are there any restrictions on processing certain data types with Bob?

The following data types should **not** be processed with Bob:

- SPI
- Personal Health Information (PHI)
- Export Regulated (ITAR, APP etc)
- Regulated Financial Data (Banking numbers, PANs, non-public financial data)

## What are the risks of using Bob?

Bob is a powerful tool, and it's important to use it responsibly. Remember:

- **Bob can make mistakes.** Always review Bob's proposed changes carefully before approving them.
- **Bob can execute commands.** Be cautious when asking Bob to run commands, especially if you're using auto-approval.

# Usage

---

## How do I start a new task?

Open the Bob panel and type your task in the chat box. Be clear and specific about what you want Bob to do.

## How do I switch between modes?

Use the dropdown menu in the chat input area to select a different mode, or use the `/` command to switch to a specific mode.

## What are tools and how do I use them?

Tools are how Bob interacts with your system. Bob automatically selects and uses the appropriate tools to complete your tasks. You don't need to call tools directly. You will be prompted to approve or reject each tool use.

## What are context mentions?

Context mentions are a way to provide Bob with specific information about your project, such as files, folders, or problems. Use the "@" symbol followed by the item you want to mention (e.g., `@src/file.ts`, `@problems`).

## Can Bob run commands in my terminal?

Yes, Bob can run commands. You will be prompted to approve each command, unless you've enabled auto-approval for commands. Be extremely cautious auto-approving commands.

## How do I provide feedback to Bob?

You can provide feedback by approving or rejecting Bob's proposed actions. You can provide additional feedback by using the feedback field.

## Can I customize Bob's behavior?

Yes, you can customize Bob in several ways:

- **Custom instructions:** Provide general instructions that apply to all modes, or mode-specific instructions.
- **Bob rules:** Create markdown rule files in your `.bob/rules/` folder in the root of project to provide additional guidelines.
- **Settings:** Adjust various settings, such as auto-approval, diff editing, and more.

## Does Bob have any auto approval settings?

Yes, Bob has a few settings that when enabled will automatically approve actions.

# Advanced Features

---

## What is MCP (Model Context Protocol)?

MCP is a protocol that allows Bob to communicate with external servers, extending its capabilities with custom tools and resources.

## Can I create my own MCP servers?

Yes, you can create your own MCP servers to add custom functionality to Bob.

# Troubleshooting

---

## Bob isn't responding. What should I do?

- Make sure your API key is correct and hasn't expired.
- Check your internet connection.
- Try restarting VS Code.

## Bob made changes I didn't want. How do I undo them?

Bob uses VS Code's built-in file editing capabilities. You can use the standard "Undo" command (Ctrl/Cmd + Z) to revert changes. You can also use checkpoints to revert changes made to a file.